# Pengantar Pengujian Perangkat Lunak

**Mata Kuliah:**
**Pengujian Perangkat Lunak (CIF62128 )**
**Disusun oleh:**
**Software Engineering Lab.**

Version: 1.0
Date: 11; Month: February; Year: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

# Agenda

- Introduksi
- Tentang Mata Kuliah
- Materi Perkuliahan
- Referensi
- Komponen Penilaian
- Materi 1. Pengantar Pengujian Perangkat Lunak


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Introduksi

- Nama: Mahardeka Tri Ananta
- Prodi: Teknik Informatika
- Laboratorium: Rekayasa Perangkat Lunak (RPL)
- Kontak: ***google chat*** → **deka@ub.ac.id** , Ruang F6.8.16
- Keminatan: *Requirements Engineering* dan *User Experience*


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Tentang Mata Kuliah

- Kredit: 3 SKS (pilihan)
- Deskripsi:
  Mata kuliah ini membahas konsep, strategi dan penerapan teknik-teknik pengujian perangkat lunak secara lebih mendalam pada proses pengembangan perangkat lunak. Bahasan mata kuliah ini meliputi: **konsep dasar pengujian perangkat lunak**, **strategi** pengujian perangkat lunak, **teknik pengujian** perangkat lunak, pengujian **fungsional** dan pengujian **non fungsional** serta ***debugging***.


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Capaian Pembelajaran Mata Kuliah

- Mahasiswa mampu **menjelaskan konsep** pengujian perangkat lunak dengan baik
- Mahasiswa mampu **menerapkan strategi** pengujian perangkat lunak
- Mahasiswa mampu **menerapkan teknik** pengujian perangkat lunak dan automasinya dengan benar


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Materi Perkuliahan

1. Pengantar pengujian PL
2. Konsep dasar pengujian PL
3. Strategi pengujian PL dan dokumentasi standar
4. Teknik pengujian PL
5. Pengujian fungsional
6. Pengujian nonfungsional
7. Debugging


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Referensi


- **R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill Education, 2020.**
- **I. Sommerville, Software Engineering, 10th ed. Pearson Education, 2016.**
- D. Graham, E. van Veenendaal, I. Evans, and R. Black, Foundations of Software Testing: ISTQB Certification, 3rd ed. Cengage Learning, 2012.
- P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge University Press, 2017.
- W. E. Perry, Effective Methods for Software Testing, 3rd ed. Wiley Publishing, Inc., 2006.
- M. Aniche, Effective Software Testing: A Developer’s Guide. Manning Publications, 2022.




© Fakultas Ilmu Komputer, Universitas Brawijaya

# Komponen Penilaian

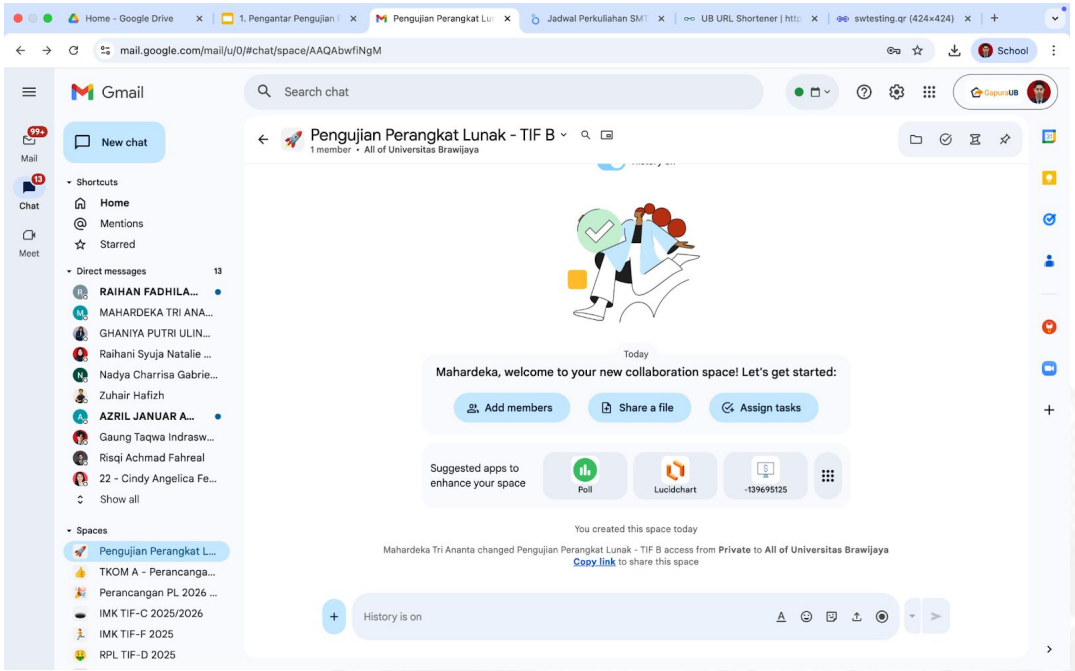
| Komponen | BOBOT (%) |
|---|---|
| Presensi (Aktivitas Partisipatif) | 5 |
| Tugas 1 | 5 |
| Tugas 2 | 5 |
| Tugas 3 | 5 |
| Kuis 1 | 15 |
| Kuis 2 | 15 |
| Ujian Tengah Semester (UTS) | 25 |
| Ujian Akhir Semester (UAS) | 25 |
| **TOTAL** | 100.00 |

**Peringatan:**

- Presensi **< 80 %** □ Nilai Akhir = **E**, baca buku pedmoan akademik UB dan FILKOM
- Tugas/Kuis/Ujian susulan dilakukan dengan alasan yang mendesak (bukan kelalaian  dan valid) dan sesuai aturan Ujian yang berlaku

# Kanal Komunikasi Kelas

- Google Space: https://s.ub.ac.id/swtesting




***Login gmail dengan akun @student.ub.ac.id

# Q&A

# Mengapa Pengujian Perangkat Lunak Penting?

**Realitas Industri:**

- 50% biaya pengembangan untuk testing
- Bug di production 100x lebih mahal
- Kegagalan sistem dapat merugikan jutaan

**Contoh Kasus:**

- Ariane 5 (1996): $370 juta
- Mars Climate Orbiter (1999): $125 juta
- Therac-25: Korban jiwa

**Manfaat Testing:**

- ✅ Mendeteksi defect lebih awal
- ✅ Meningkatkan kualitas produk
- ✅ Mengurangi biaya maintenance
- ✅ Membangun kepercayaan pengguna
- ✅ Memenuhi standar industri


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Evolusi Pengujian Perangkat Lunak

**Era Traditional (1950-1980s)**

- Manual testing dominan
- Testing = finding bugs
- Testing di akhir development

**Era Structured (1990s-2000s)**

- Metodologi testing terstruktur
- Test planning & documentation
- Testing sebagai fase terpisah

**Era Agile & Modern (2000s-now)**

- Test-driven development (TDD)
- Continuous integration/testing
- Shift-left testing approach
- Test automation

**Era DevOps & CI/CD (2010s-now)**

- Testing as code
- Pipeline automation
- Quality gates


© Fakultas Ilmu Komputer, Universitas Brawijaya

# International Software Testing Qualifications Board (ISTQB)

- Organisasi internasional non-profit yang bertanggung jawab untuk standarisasi kualifikasi dan sertifikasi dalam bidang software testing.
- Organisasi global yang didirikan tahun 2002
- Standar internasional untuk kualifikasi software testing
- Badan sertifikasi yang diakui di lebih dari 120 negara
- Penyusun kurikulum dan silabus testing yang terstandarisasi


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Fundamental Testing Concepts

**7 Prinsip Dasar Testing (ISTQB)**

1. **Testing shows presence of defects** - Testing dapat menunjukkan adanya defect, tapi tidak dapat membuktikan tidak ada defect
2. **Exhaustive testing is impossible** - Testing menyeluruh tidak mungkin dilakukan
3. **Early testing** - Testing harus dimulai sedini mungkin
4. **Defect clustering** - Sebagian besar defect ditemukan di modul tertentu
5. **Pesticide paradox** - Test case yang sama tidak akan menemukan bug baru
6. **Testing is context dependent** - Testing bergantung pada konteks aplikasi
7. **Absence of errors fallacy** - Tidak adanya error tidak berarti sistem berkualitas

# Test Levels & Types Overview

**Test Levels:**

- **Unit Testing** - Individual components
- **Integration Testing** - Component interactions
- **System Testing** - Complete system
- **Acceptance Testing** - User requirements

**Test Types:**

- **Functional Testing** - What the system does
- **Non-functional Testing** - How the system performs

**Test Approaches:**

- **Static Testing** - Review & analysis
- **Dynamic Testing** - Executing code

**Test Techniques:**

- **Black Box** - Specification-based
- **White Box** - Structure-based
- **Grey Box** - Combination approach


© Fakultas Ilmu Komputer, Universitas Brawijaya

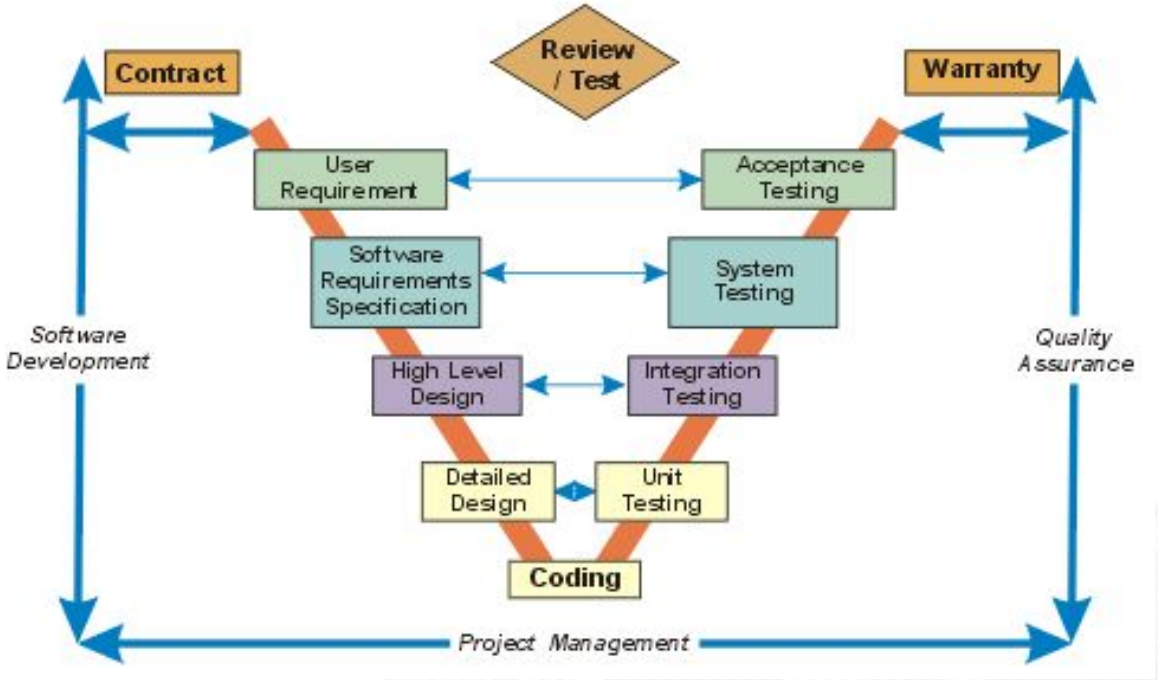
# Strategi Pengujian dalam SDLC

Strategi pengujian berubah mengikuti model pengembangan yang digunakan (Sommerville, 2016):

- **Waterfall Model:** Pengujian adalah fase diskrit di akhir. Berisiko tinggi jika kesalahan ditemukan pada tahap arsitektur.
- **Incremental Development:** Pengujian dilakukan pada setiap *increment* (potongan fitur). Memudahkan identifikasi masalah pada fitur baru sebelum digabung ke sistem utama.
- **V-Model:** Menekankan hubungan antara fase pengembangan (kiri) dengan fase pengujian yang sesuai (kanan).


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Testing dalam Software Development Life Cycle




- **Requirements Acceptance Testing**: Validasi kebutuhan bisnis.
- **System Design System Testing**: Menguji integrasi seluruh komponen.
- **Architecture Design Integration Testing**: Menguji antarmuka antar modul.
- **Coding Unit Testing**: Verifikasi logika internal kode terkecil.

**Key Insight:** Setiap fase development memiliki aktivitas testing yang bersesuaian

# Pengujian dalam Agile & DevOps

Dalam lingkungan modern (Agile), pengujian bersifat *pervasive* (menyerap di semua tahap):

- **Test-Driven Development (TDD):** Menulis tes sebelum menulis kode (Sommerville menyebutnya sebagai cara untuk meningkatkan kepercayaan diri pada kode).
- **Continuous Integration (CI):** Pengujian otomatis dijalankan setiap kali ada perubahan kode kecil.
- **Automated Regression Testing:** Memastikan fitur lama tidak rusak karena adanya pembaruan fitur baru.

# Verifikasi vs Validasi (V&V)

Sommerville mendefinisikan V&V sebagai proses pemeriksaan apakah sistem sudah sesuai tujuan:

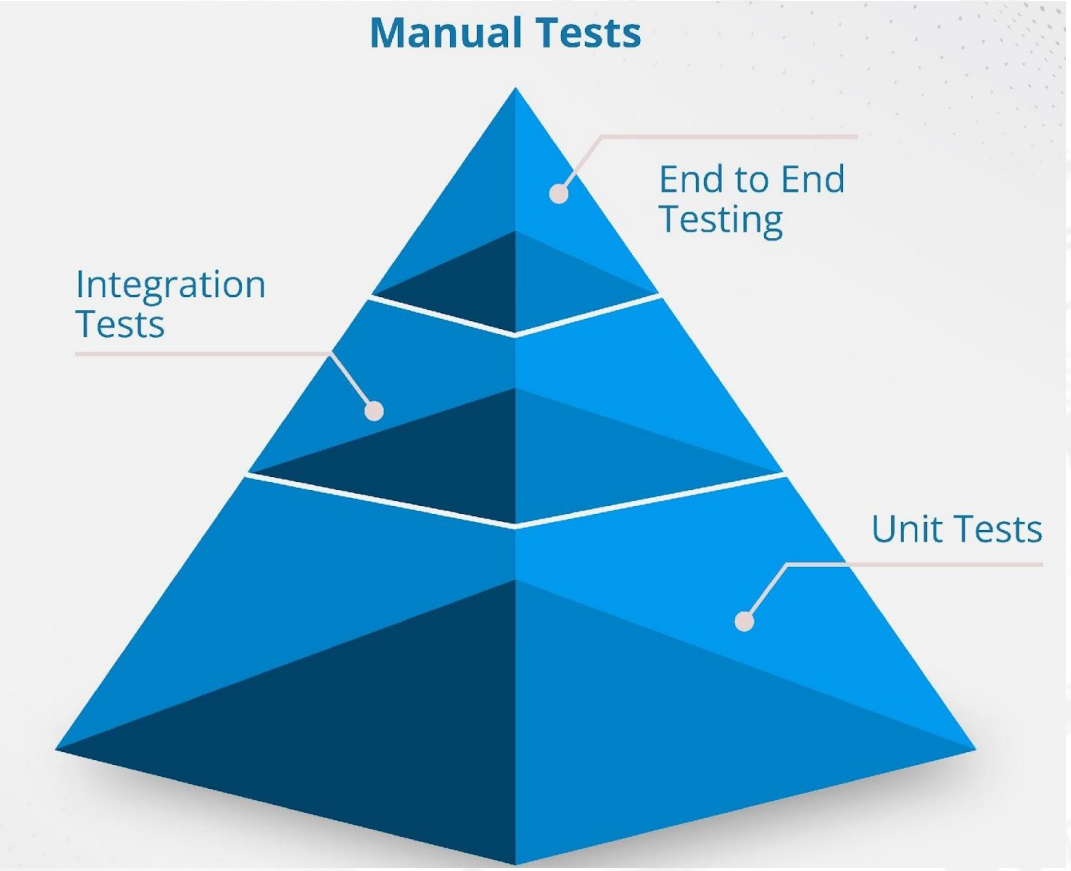
- **Verification (Kesesuaian Spesifikasi):** "Are we building the product right?"
  - **Analogi Kacamata:** Pabrik membuat kacamata minus 2 sesuai pesanan dokter. Secara teknis sempurna.
- **Validation (Kebutuhan Pengguna):** "Are we building the right product?"
  - **Analogi Kacamata:** Ternyata pasien sebenarnya butuh minus 5. Maka kacamata tersebut **tidak valid**.

**Contoh Software (Ojek Online):**

- **Verifikasi:** Fitur GPS menampilkan koordinat dengan benar (Teknis OK).
- **Validasi:** Aplikasi tidak menyediakan opsi "Pilih Titik Jemput di Lobi Gedung", sehingga driver bingung (Kebutuhan Pengguna tidak terpenuhi).


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Test Automation Pyramid




- **Unit Tests:** Fast, isolated, banyak
- **Integration Tests:** Medium speed, component interaction
- **E2E Tests:** Slow, full user journey, sedikit

# Modern Testing Tools & Technologies

**Unit Testing:**

- JUnit (Java), pytest (Python)
- Jest (JavaScript), NUnit (.NET)

**Integration Testing:**

- TestContainers, Postman
- REST Assured, WireMock

**E2E Testing:**

- Selenium, Cypress
- Playwright, TestCafe

**Performance Testing:**

- JMeter, K6, Gatling
- LoadRunner

**Security Testing:**

- OWASP ZAP, Burp Suite
- SonarQube

**CI/CD Integration:**

- Jenkins, GitHub Actions
- GitLab CI, Azure DevOps


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Quality Assurance vs Quality Control

**Quality Assurance (QA)**

- **Fokus:** Proses
- **Tujuan:** Mencegah defect
- **Aktivitas:**
  - Process improvement
  - Standards compliance
  - Training & guidelines
- **Sifat:** Proactive

**Quality Control (QC)**

- **Fokus:** Produk
- **Tujuan:** Mendeteksi defect
- **Aktivitas:**
  - Testing & validation
  - Reviews & inspections
  - Defect reporting
- **Sifat:** Reactive

**<u>Testing</u> adalah bagian dari QC, sedangkan QC adalah bagian dari QA**


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Tantangan dalam Testing Modern

**Technical Challenges:**

- Microservices complexity
- API testing
- Mobile & cross-platform
- Cloud & containerization
- AI/ML system testing

**Organizational Challenges:**

- Skill gap & training
- Tool selection
- ROI measurement
- Cultural resistance
- Resource allocation

**Process Challenges:**

- Agile & fast delivery
- Continuous integration
- Test data management
- Environment consistency

**Emerging Trends:**

- AI-powered testing

# Roadmap Pembelajaran

**Journey Mata Kuliah PPL**

**Phase 1:** Foundation → Memahami konsep & prinsip dasar

**Phase 2:** Strategy → Merancang strategi testing yang efektif

**Phase 3:** Techniques → Menguasai teknik testing (functional & non-functional)

**Phase 4:** Practice → Implementasi & debugging dalam proyek nyata

**Goal:** Menjadi *Software Tester* yang kompeten!


© Fakultas Ilmu Komputer, Universitas Brawijaya

# Kesimpulan

- **Pengujian** adalah disiplin sistematis (Verifikasi & Validasi) untuk memastikan kualitas.
- **Kualitas** dipengaruhi oleh aspek teknis (kode) dan aspek manusia (kebutuhan pengguna).
- **Urgensi** pengujian mencakup aspek ekonomi (biaya perbaikan), keselamatan (nyawa), dan reputasi.
- **Strategi** terbaik adalah mengintegrasikan pengujian sejak awal siklus pengembangan (*Shift-Left Testing*).


© Fakultas Ilmu Komputer, Universitas Brawijaya